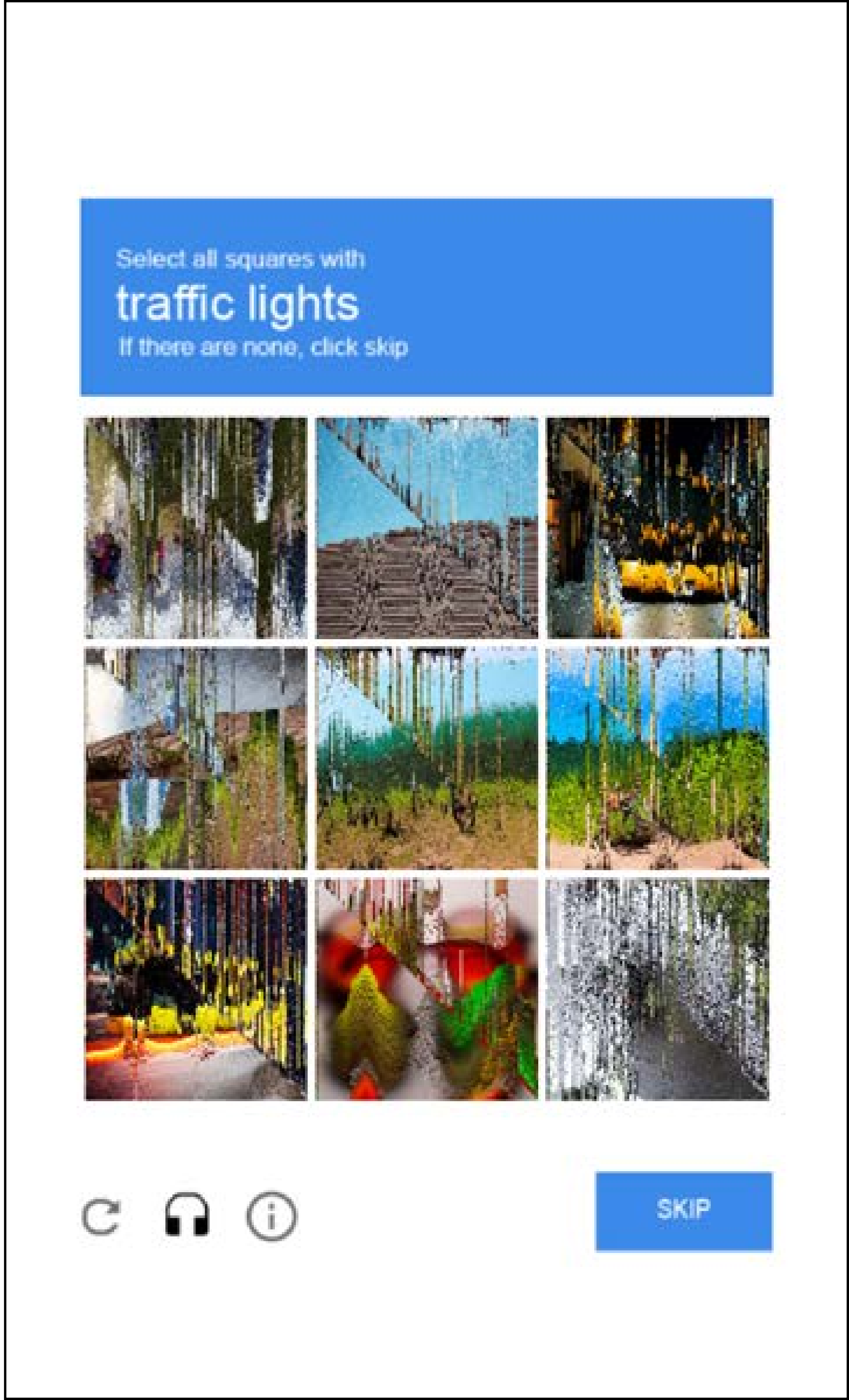
Final Showcase

CAPTCHA

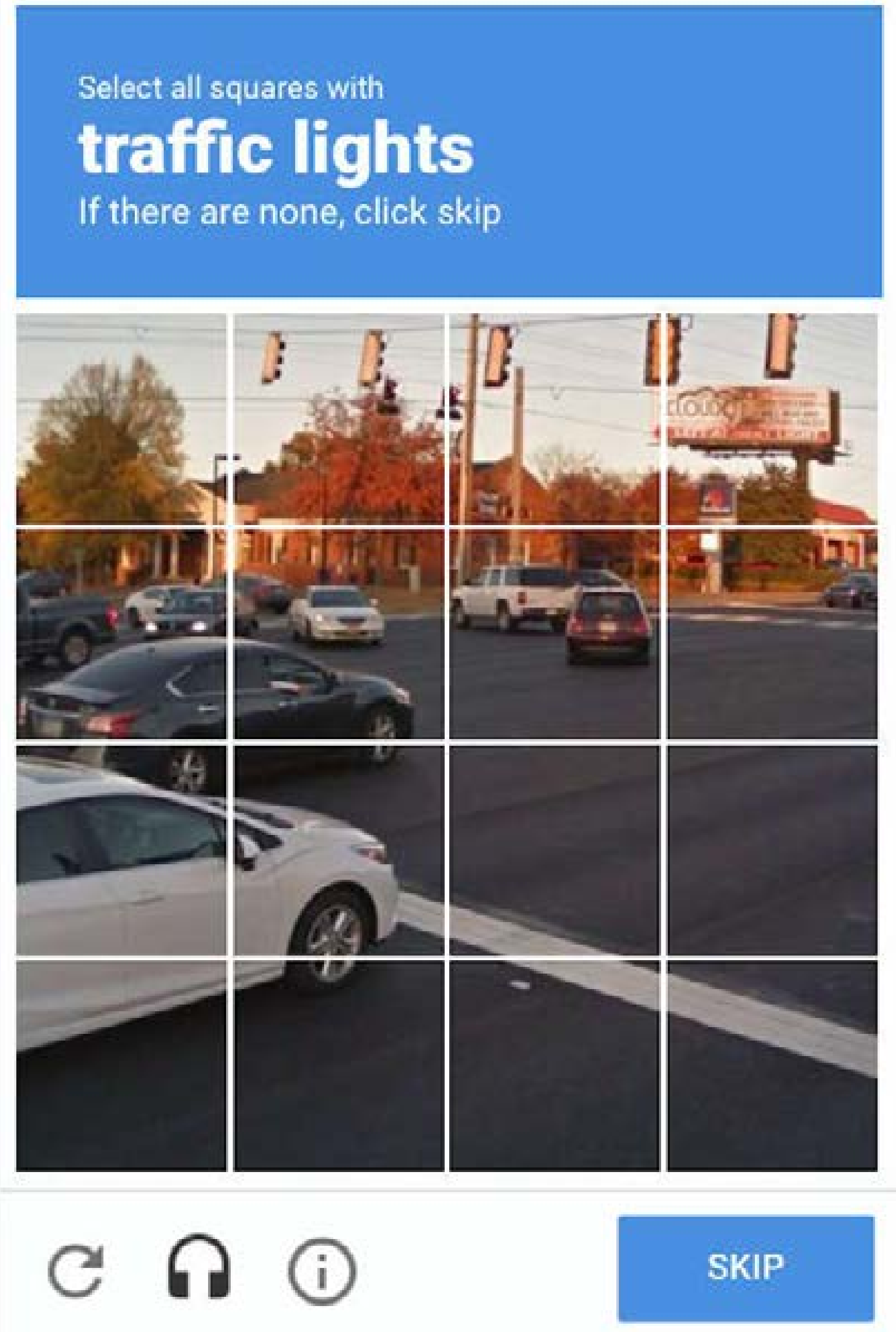
By: Amy (Eunjong) Lee

UAL Summer Study Abroad: Creative Computing



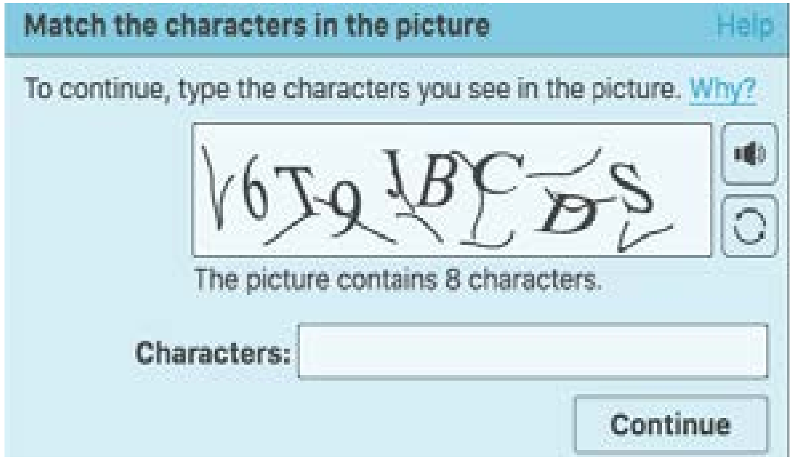
IDEA

- Why do CAPTCHAs expect <humans> to know what <traffic lights> are?
    - How come the criteria to differentiate between <human> and <bots> become <traffic light>?
- What if a user has never seen a <traffic light> before?
    - Even buildings differ from culture to culture: some places have more angular buildings while other places have more circular architecture
- What if they come from a place without <traffic light>?
    - Based on the assumption that those with access to computers must have seen a <traffic light> before



CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart)

- Type of challenge-response Turing test
- term coined in 2003 by Luis von Ahn, Manuel Blum, Nicholas J. Hopper, John Langford
    - von Ahn and Blum created a program that generates random string of text, generates distorted image of that text, displays image, asks user to enter text, require user to click “I am not a robot” checkbox



- Used to block bots and activated when activity seems suspcious
- Based on reading texts or images making it difficult for blind, visually impaired users
- Other disadvantages include bad user experience, advanced bots beating the system
- Google’s reCAPTCHA
    - v1: images from Google Street View; proves humanity through real-world object identification
    - v2: replace text and image-based challenges with “I am not a robot” tracking mouse movements
    - v3: score system that identifies what bots are doing and where within the website, no interruption

Idea 1: A series of CAPTCHA gradually getting “harder” to solve due to image distortion

- Like a game with “levels” that get more difficult
- Kind of “spoon feeds” audience story artwork is telling

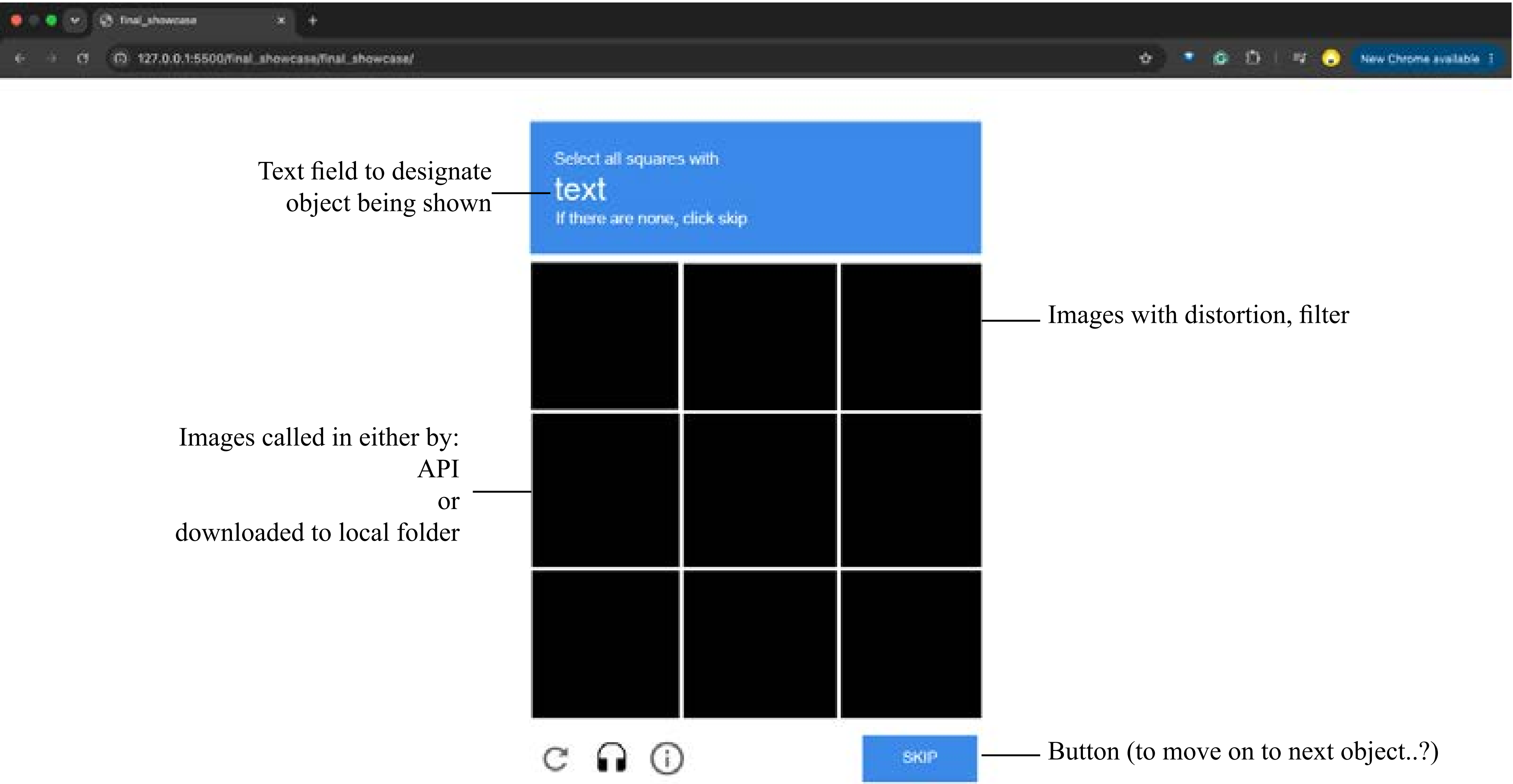
Idea 2: A single CAPTCHA with type interaction to designate object for image recognition

- Best tells the story (I think)
- Would be most difficult as an API would be needed to retrieve whatever objects user entered in

Idea 3: A single CAPTCHA where image pixels scatter with mouse movements

- Most “fun” user experience
- Strays away from story artwork is trying to depict

**Sketchbook thumbnails can be found in Day 8 Reflections in READ.ME



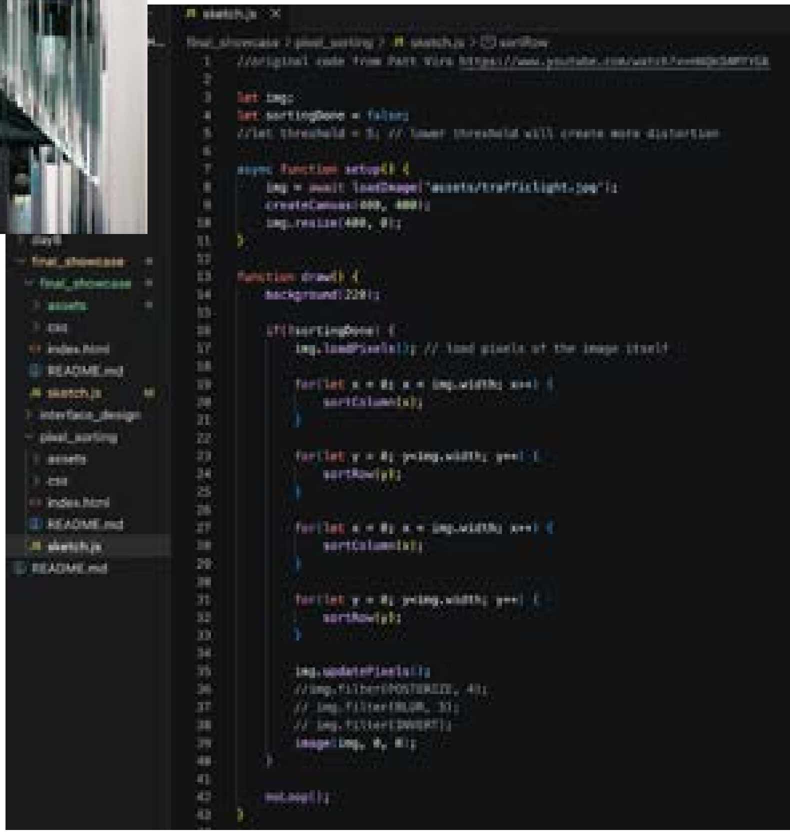
Text field to designate object being shown

Images with distortion, filter

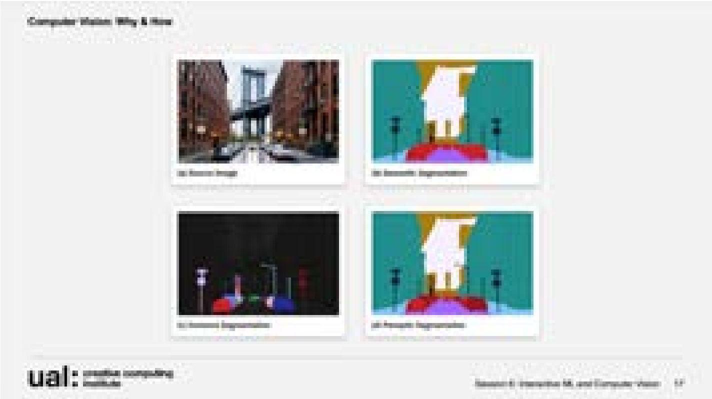
Images called in either by:
API
or
downloaded to local folder

Button (to move on to next object..?)

Pixel Sorting for Image Distortion



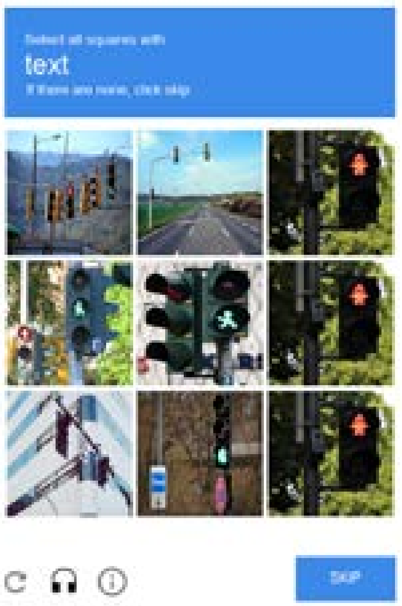
- I studied pixel sorting and an example code from online to utilize in my final
- I have a separate code in my GitHub named pixel_sorting that has the code I was studying and experimenting with
- To the right is the image I used pixel sorting on
- Below are class lectures that I was inspired by: I wanted to make some filter or image manipulating code to distort the image so that it would be hard to tell what it is
- I spent Day 9 exploring APIs (that I ended up not using), pixel sorting and other image manipulations (glitch art, glitch shader), and making graphics in Adobe Illustrator



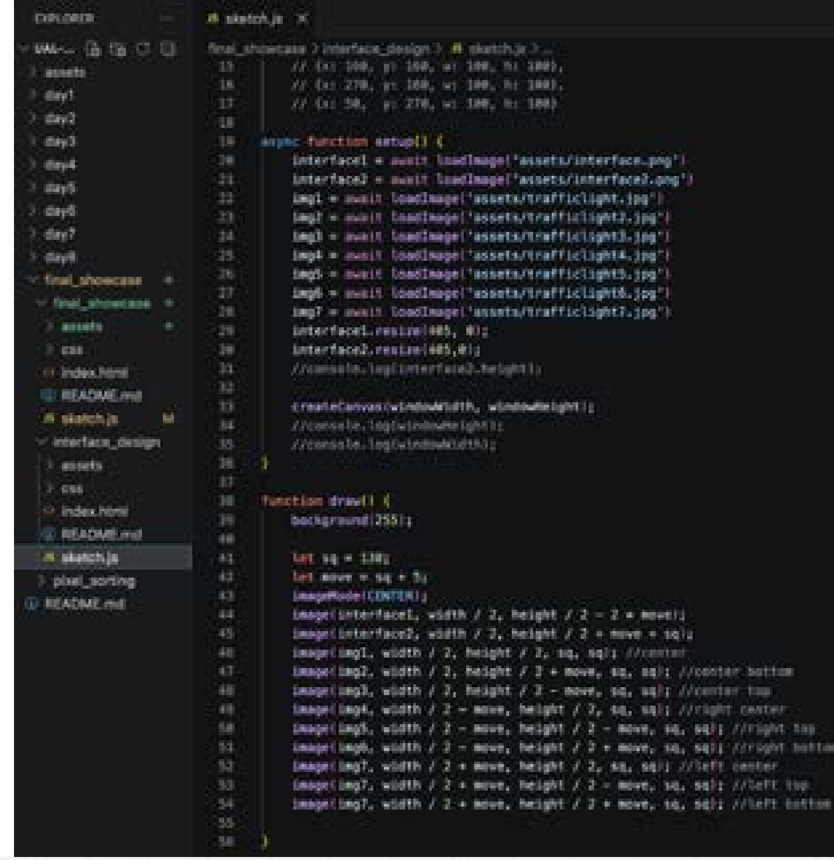
DAY 10: PROGRESS

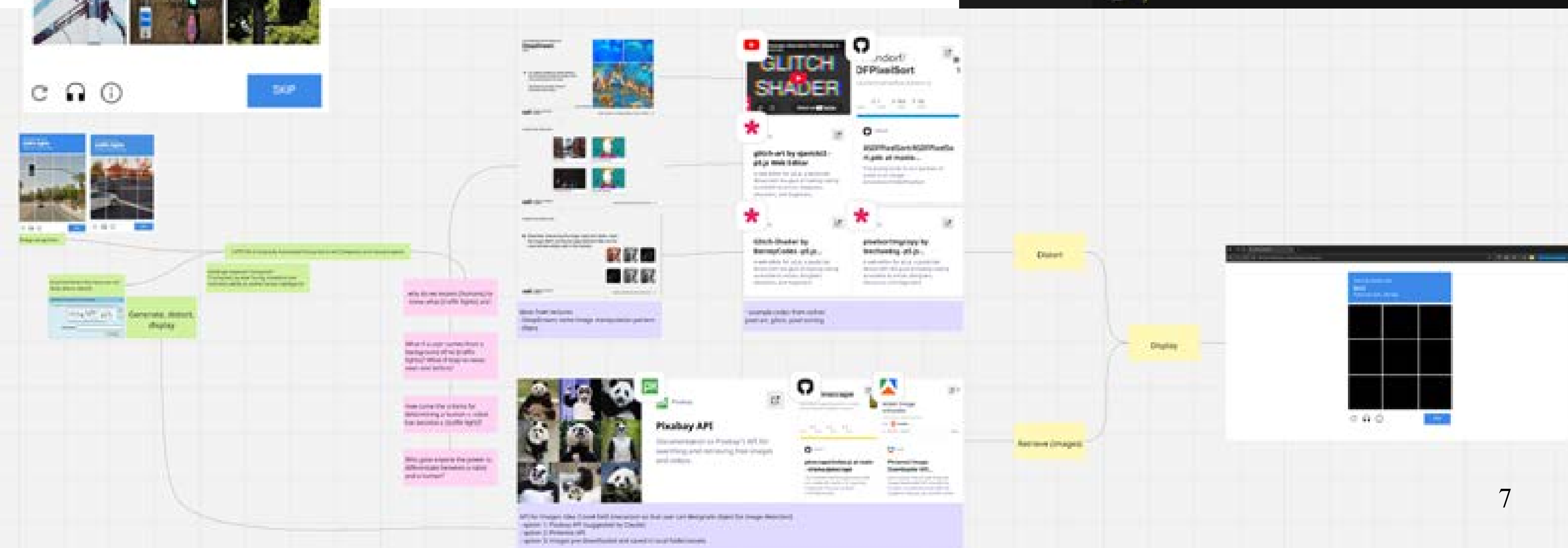
CAPTCHA Interface with Custom Graphics

- I spent the day combining pixel_sorting & interface_design and completing my Miro Board to plan whether I wanted to use APIs or something else



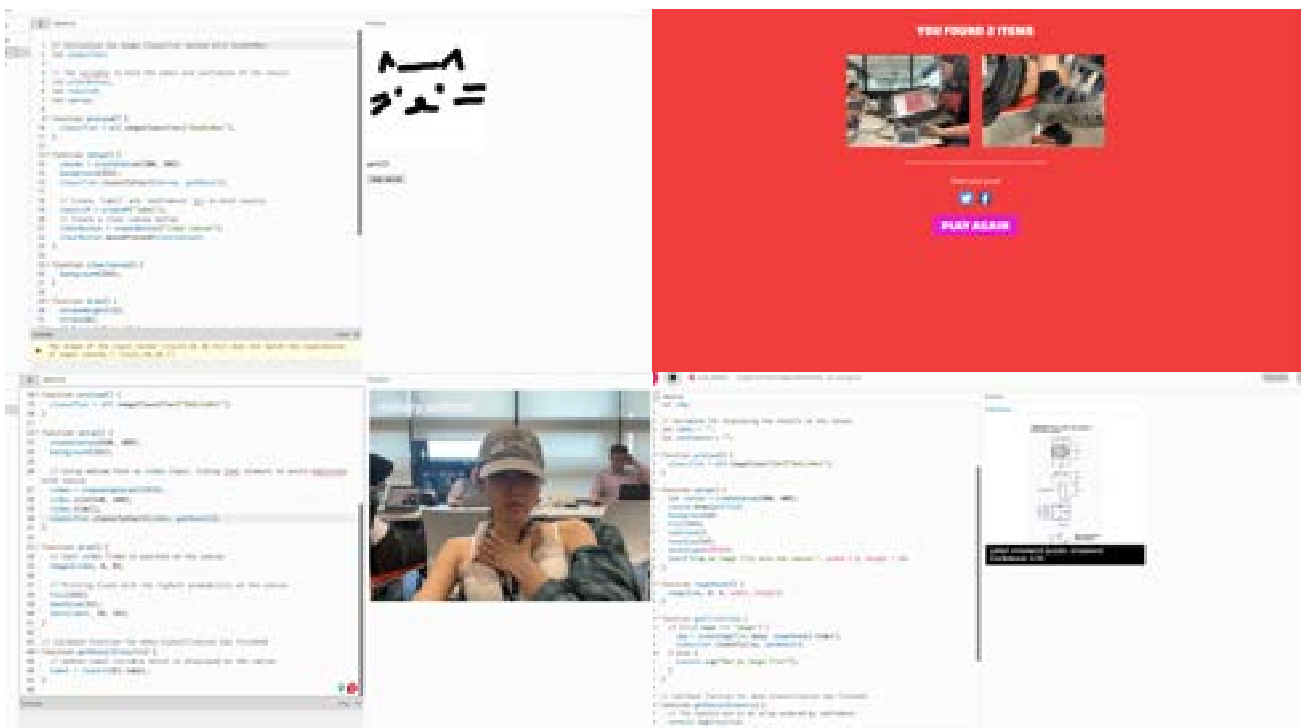
- To the left is a draft of my final: this took longer than expected as you can see me calculating and adjusting the grid with equations like width / 2 + move, etc.
- I also worked on my reflections and this documentation to refreshen myself





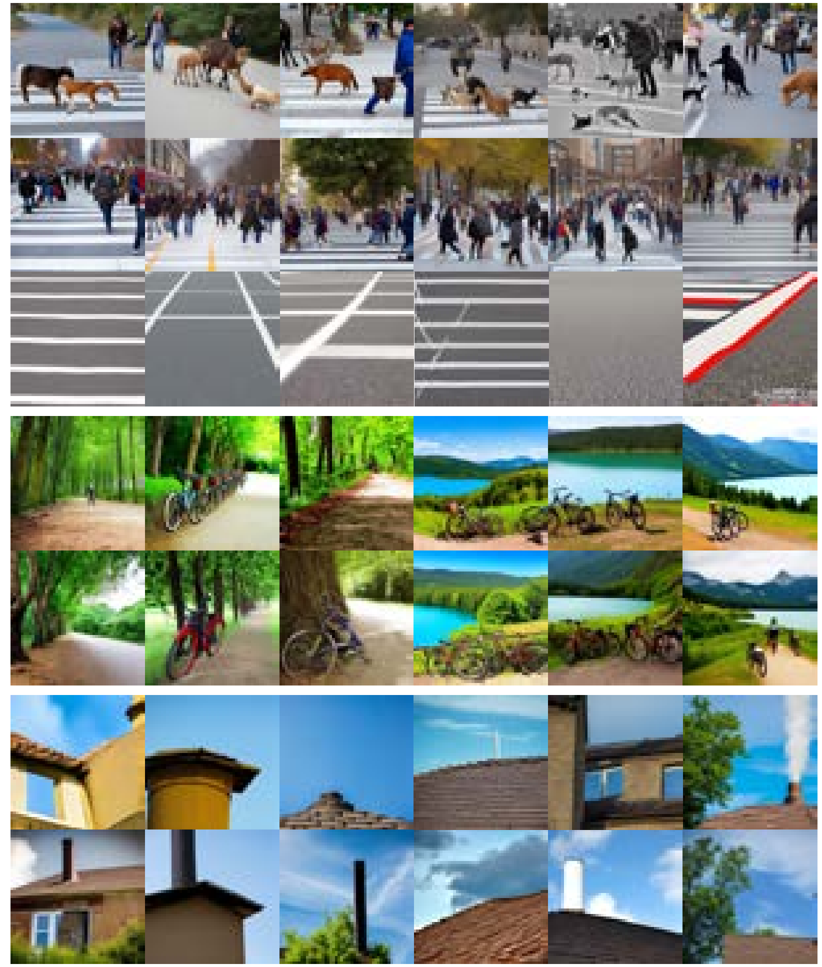
Adding Functionality and Adjusting Visuals

- I added a button to allow users to use the SKIP button graphic and click on images to change them: like a real CAPTCHA
- I had a lot of problems because the SKIP button was part of a graphic I predesigned in Illustrator so I had to match the button coordinate, tried playing with layers in p5.js, etc.
- I also wanted to experiment more with the images: I ended up adjusting the pixel sorting code to make it more original while also making the images more dynamic and producing my own images with ComfyUI

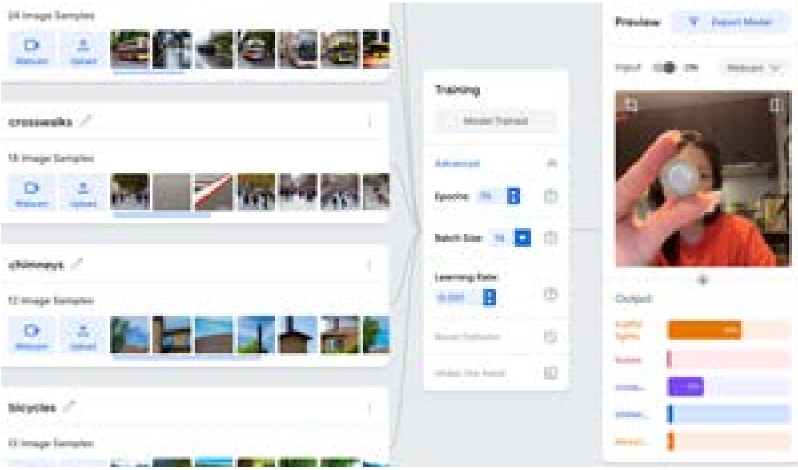


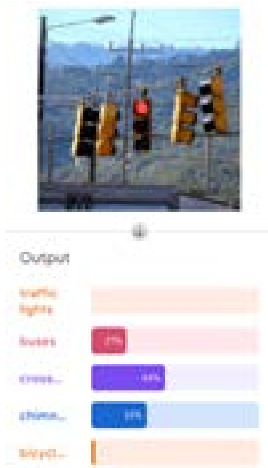
Classwork Inspiration

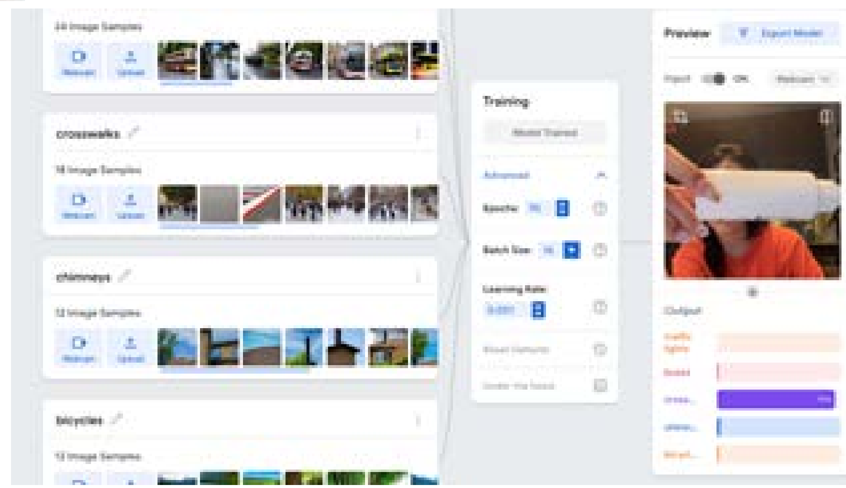
- I was looking back at the classworks while working on the READ.ME and my reflection from this exploration inspired me
- I also wanted to somehow incorporate other thing we spent time in class than just p5.js coding



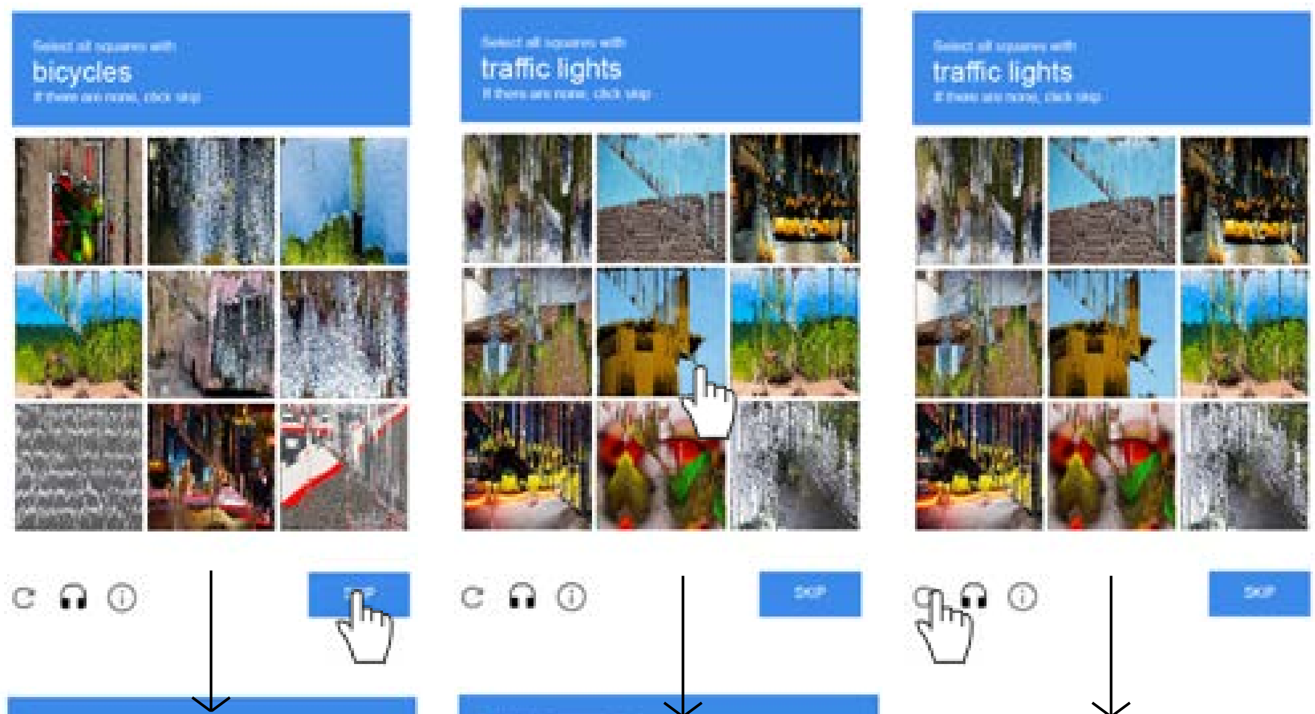
- These are the images I produced on ComfyUI
- I wanted to elaborate on the process of communicating with the machine or AI, so I used prompts like “a Korean bus...”, or “a British bus...” with negative prompts for flags or symbols; “traffic lights in circular shapes...” or “traffic lights in angular shapes...”
- Because the concept of this work is also based on how different cultures have different visuals of even the same object, I wanted to explore this area with AI

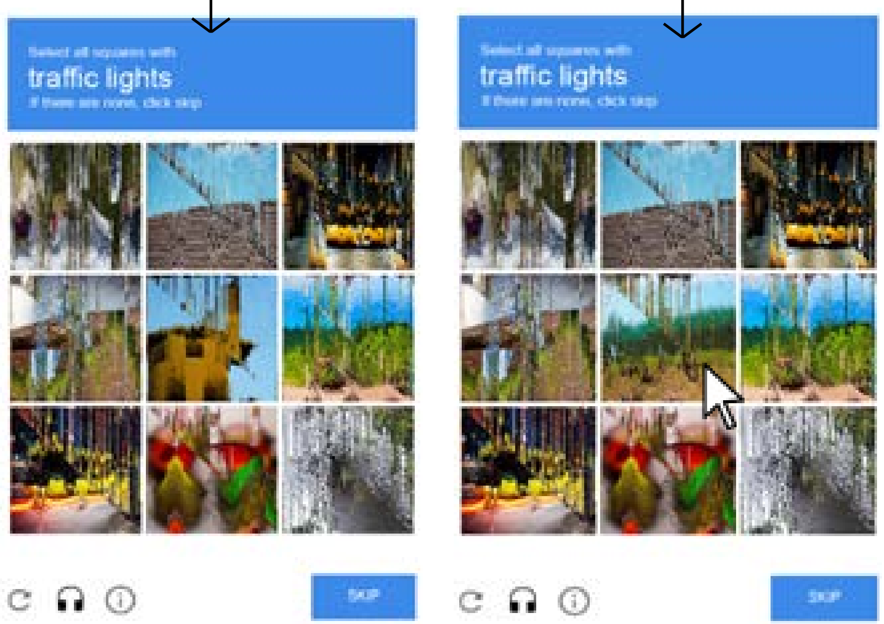






- I fed the images I produced in ComfyUI to train a model in Teachable Machine, then I used the webcam and the images I had found on Pixabay to interact with the model
- Unfortunately, I couldn’t come up with a way to incorporate this experiment in the final showcase also because this idea came kind of late
- I still feel that this experimentation is important to the final because it adds more meaning and strengthens the purpose of me questioning a CAPTCHA and telling humans and robots apart





CAPCTHA

Amy Eunjong Lee

400 lines of p5.js, 37 images produced on ComfyUI, Adobe Illustrator

Clicking the SKIP button to change objects for recognition or the images to attempt to solve the puzzle, you can find yourself stuck in this infinite CAPTCHA.

Traffic lights, chimneys, crosswalks, and bicycles are objects expected to be common enough for humans to recognize them by images. What happens if you are a human who has never seen them before?

http://www.captcha.net/
https://en.wikipedia.org/wiki/CAPTCHA
https://www.cloudflare.com/learning/bots/how-captchas-work/
https://developers.google.com/recaptcha
https://www.ibm.com/think/topics/captcha
https://editor.p5js.org/leechaeing/sketches/r7v9spl8b
https://glitchology.com/pixel-sorting/
https://teachablemachine.withgoogle.com/models/uIqrTEqW0/
https://github.com/kimasendorf/ASDFPixelSort/blob/master/ASDFPixelSort.pde

**all included and explained in GitHub READ.ME